# AICE Professional
# n수생 탈출기

여러분은 저 같은 **시행착오**를 겪지 않도록 준비했습니다. Tabular · Text · Image 를 시험장에서 바로 쓰는 순서와 준비물로 정리했어요.

실기 180분  Colab T4  발표 45분 + Q&A

→ 다음  ← 이전  0 목차

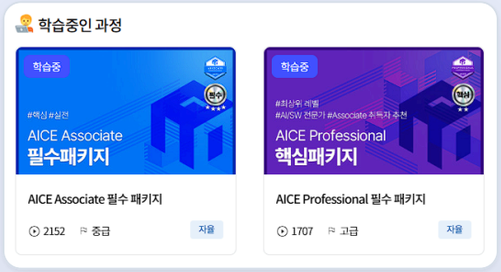
참고자료

# 이 과정과 함께 볼 네 가지

강의자료

## KT 학습플랫폼 지니어스

AICE Associate·Professional 과정 — ktedu.kt.com




공식 샘플 문항

## AICE Pro 샘플 문항

aice.study/certi/practice

**Associate·Professional 문항을 모두 참고**하세요.

시험 가이드

## AICE 응시 가이드

aice.study/certi/examGuide

시험 환경·제출 규칙·유의사항을 공식 문서로 미리 확인하세요.

연습 환경

## Colab

colab.research.google.com

**런타임 → 런타임 유형 변경 → T4 GPU**로 맞춰 연습하세요. 시험장(T4급 GPU)과 비슷한 환경입니다.

이 시험의 성격

# 저는 모델보다 절차에서 더 많이 틀렸어요

> **정확도 올리기 전에**
> **끝까지 도는 코드부터 만들어요.**

train만 전처리하고 test엔 빠뜨림 · 스케일러를 test로 fit(train으로만 해야) · 제출 컬럼명·값이 지문과 다름 — 실력과 상관없이 **점수를 잃는 곳**이 대부분 여기예요.

시험 환경

# 실력과 상관없이 놓치기 쉬운 여섯 가지

01 · 자료

### 분석 자료가 주어진다

먼저 읽으세요. 누수 컬럼이 거기서 드러납니다.

02 · GPU

### 자동으로 안 붙는다

시작할 때 한 번, **튕기면 다시.** 재연결 시 CPU로 붙기도 합니다.

03 · 저장

### 중간저장은 코드만

변수와 GPU는 안 지켜집니다. 학습 결과는 **파일로** 따로.

04 · 제출

### 휴대폰번호_문제번호

`01012345678_1.csv`

하이픈 없이 붙여 씁니다.

05 · LABEL

### 값을 label로 복원

인코딩 숫자 그대로 내면 **0점.**

컬럼명도 요구 그대로.

06 · 커널

### 커널이 하나다

학습이 긴 문제부터. 변수는 `p1_` · `p2_` · `p3_` 로 분리.

마지막 차시에서 다시 다룰게요.

이 과정의 뼈대

# 여덟 단계, 세 도메인 공통

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|
| **로드** | **EDA** | **전처리** | **분할** | **모델 비교** | **튜닝** | **평가** | **제출** |

| 바뀌는 단계 | TABULAR | TEXT | IMAGE |
|---|---|---|---|
| **3 · 전처리** | 결측 · 인코딩 · 스케일링 | 정제 → 벡터화(TF-IDF) | 리사이즈 → 증강 |
| **5 · 모델** | RF · GB · LogReg | LinearSVC · LogReg · NB | MobileNetV2 |

나머지 **여섯 단계는 코드가 그대로** 재사용됩니다. 노트북 네 개가 닮아 보인다면 실수가 아니라 의도입니다.

준비 전략

# 노트북 4본으로 끝내는 AICE Pro

문제 1 · TABULAR · 30점

**분류**

이진/다중 자동 대응

문제 1 · TABULAR · 30점

**회귀**

모델·지표만 다르고 뼈대는 같다

문제 2 · TEXT · 35점

**다중분류 한 벌**

이진용을 따로 만들지 않는다

문제 3 · IMAGE · 35점

**다중분류 한 벌**

이진용 없음 · 클래스 수와 무관하게 한 벌

> **분류는 모두 다중분류로 통일.**
> **다중분류에 클래스가 2개면 이진분류.**

**실습은 모두 Colab T4 (무료 GPU) 환경에서 진행합니다.** 시작할 때 `런타임 → 런타임 유형 변경 → T4 GPU`.

**배점: Tabular 30 · Text 35 · Image 35 = 100점.**
시험 순서는 **1번 Tabular · 2번 Text · 3번 Image**로 고정 — 파일명과 변수 접두사(`p1_` · `p2_` · `p3_`)를 이 순서에 맞춰 쓰면 됩니다.

이것만은 꼭

# 세 가지 습관

습관 1

## 시드를 고정한다

```
RANDOM_SEED = 42
```

점수가 매번 달라지면 개선 여부를 판단할 수 없습니다.

습관 2

## train은 fit,<br>test는 transform

```
fit_transform(X)
transform(X_test)
```

test로 fit하면 점수가 **오히려 올라** 발견이 어렵습니다.

습관 3

## 이진/다중을<br>분기하지 않는다

```
NUM_CLASSES = W
  df[TARGET].nunique()
```

"아 3개짜리네" 하고 코드 고치는 순간 5분이 날아갑니다.

# Tabular

코드가 어렵다기보다 **정할 게 많은** 도메인.
그 결정들을 고민이 아니라 규칙으로 바꿉니다.

TABULAR · EDA

# 산점도 말고, 표 하나

```
pd.DataFrame({
    'miss_pct':    (df.isnull().mean()*100).round(1),
    'nunique_pct': (df.nunique()/len(df)*100).round(1),
    'dtype':        df.dtypes,
})
```

EDA의 목적은 감상이 아니라 **어느 컬럼을 버릴지 정하는 것**이에요.

| 신 호 | 정 체 | 조 치 |
|---|---|---|
| 결측 ≥ 70% | 정보 부족 | **제거** |
| 고유값 ≥ 95% | ID · 이름 | **제거** |
| 타겟상관 ≥ .95 | **누수 의심** | 확인 |
| 의미 중복 | class vs pclass | **하나만** |

> **정확도가 0.99를 넘으면**
> **누수부터 의심해요.**

**제거하는 건 상관이 "가짜·못 쓰는" 경우입니다** — ① **타겟 파생·복사**(타겟을 옮겨 적거나 타겟으로 계산한 값) · ② `id` **·정렬 부산물**(train만 타겟 순으로 정렬돼 생김 → test엔 안 이어짐) · ③ **사후값**(결과가 난 뒤 기록돼 예측 시점엔 못 얻는 값). 반대로 test 에도 정상인 **진짜 피처면 남깁니다.**

TABULAR · 전처리

# 순서만 외워두면 매번 안 헤매요

| a | b | c | d | e | f |
|---|---|---|---|---|---|
| 제거 | 분리 | 결측 | (이상치) | 인코딩 | 스케일링 |

| | LABELENCODER | ONEHOTENCODER |
|---|---|---|
| **트리 모델** | **충분함** | 차원만 늘어남 |
| **선형 모델** | 없는 순서를 만듦 | **적합** |

시험 주력은 트리 모델 → **LabelEncoder가 기본값.** 단, test에만 있는 값은 `-1`로 보내 에러를 막습니다.

TABULAR · 모델 선택

# 일단 다 돌려요.
# 대신 CV를 함께 봐요

| 모델 | ACCURACY | CV | |
|---|---|---|---|
| GradientBoosting | 0.818 | 0.819 | 신뢰 |
| RandomForest | 0.811 | 0.786 | 운이 좋았을 뿐 |

왜 둘 다

### Accuracy는 한 번,
### CV는 다섯 번

둘이 벌어지면 val 분할이 유리하게 걸린 것입니다. **CV가 높은 쪽**을 고르세요.

튜닝은 **1위 하나만.** 5개를 전부 GridSearch하면 시간이 안 남아요.

TABULAR · 회귀

# 전처리·분할은 그대로, 세 군데만 바꿔요

| | 분류 | 회귀 |
|---|---|---|
| **모델** | `...Classifier` | `...Regressor` |
| **분할** | `stratify=y` | 없음 (연속값) |
| **지표** | Accuracy · F1 · AUC | RMSE · MAE · $R^2$ |

RMSE

### 큰 오차에 민감

오차를 **제곱→평균→√**. 큰 실수 하나를 크게 벌줍니다. 단위는 타겟과 같음 · **작을수록 좋음.**

MAE

### 평균 오차 그대로

오차 **절댓값의 평균**. 모든 오차를 동등하게 봐 이상치에 덜 흔들림 · **작을수록 좋음.**

$R^2$

### 얼마나 설명하나

타겟 분산을 모델이 설명한 비율. **1이면 완벽**, 0이면 평균 예측 수준, 음수면 그보다 못함 · **1에 가까울수록 좋음.**

**RMSE ≫ MAE**면 큰 오차가 있다는 신호 → 이상치 처리나 로그 변환. 타겟이 치우쳐 있으면 `log1p`로 학습하고 `expm1`로 되돌립니다.

TABULAR · 문항 형태

# 모델 구조는 지시하지 않아요

정형 문항은 **데이터 파일 · 데이터 설명 · 제출 방법**만 줍니다. "히든 64→32, sigmoid" 같은 구조 지시는 없어요 — 모델·전처리는 **우리가 정합니다.**

주어지는 것

### 데이터 + 설명

컬럼 의미·타겟이 문서로 제공.

지정되는 것

### 결과 컬럼명 · 값

csv 의 결과 컬럼명과 값(예: `Y/N`)이 명시됨 — 그대로 저장.

우리가 정함

### 모델 · 전처리

트리 앙상블 비교 → best. 자유롭게.

그래서 정형은 **분류·회귀 완성 코드 한 벌**이면 충분합니다. (딥러닝 구조 지시 없음)

# Text

한국어 짧은 문장엔 **TF-IDF 한 벌**이면 충분해요. 핵심은 단어가 아니라 **문자 n-gram**입니다.

TEXT · 전략

# TF-IDF 한 벌로 끝나요

| a | b | c | d |
| --- | --- | --- | --- |
| 정제 | char_wb 벡터화 | 선형모델 비교 | best 저장 |

왜 충분한가

### 짧은 한국어에 강해요

댓글·상품명처럼 짧은 문장은 문자 n-gram TF-IDF 로 충분히 잡힙니다.

왜 안전한가

### 다운로드·OOM 없음

사전학습 모델을 안 받으니 시간초과·메모리 사고가 없어요. 금방 끝나 제출부터 확보됩니다.

왜 단순한가

### 라벨 인코딩 불필요

sklearn 은 문자 라벨을 그대로 학습·예측 — 숫자로 바꿨다 되돌리는 단계가 없습니다.

이진이든 다중이든 **코드는 한 벌.** 클래스가 2개면 그게 곧 이진분류예요.

TEXT · 이 차시의 핵심

# 단어로 자르면 흩어져요
# 문자 n-gram으로 잡아요

"영화가" "영화는" "영화를" "영화도" → 단어로 자르면 전부 다른 피처로 흩어진다

```
# 형태소 분석기는 시험 환경에 없을 수 있다. 대신 문자 단위로:
TfidfVectorizer(analyzer='char_wb', ngram_range=(2, 4), max_features=50000)
```

왜 되나

### 어미·띄어쓰기에 안 흔들려요

"영화가 / 영화는"이 `영화` 라는 부분문자열을 공유합니다. 조사가 바뀌어도 같은 신호로 잡혀요.

파라미터

### (2,4)면 충분

2~4글자 조합에 `max_features` 만 넉넉히. 여기서 크게 더 짜낼 건 없어요.

> **모델을 바꾸기 전에 analyzer를 바꿔 보세요.**

# Image

고민할 건 제일 적어요. 대신 **레시피만 정확히** 따라가요.

IMAGE · 학습 구조

# Phase 2는 학습률을 100배 낮추는 게 전부예요

PHASE 1 · FEATURE EXTRACTION

### base 동결 · lr = 1e-3

ImageNet이 아는 특징은 그대로 두고 분류 헤드만 학습.

PHASE 2 · FINE-TUNING

### 상위 30층 해제 · lr = 1e-5

학습률이 높으면 ImageNet 가중치가 **첫 배치에 파괴**됩니다.

```
base_model.trainable = True
for layer in base_model.layers[:-30]: layer.trainable = False

model.compile(optimizer=Adam(1e-5), ...)   # trainable 바꿨으면 반드시 재컴파일
```

IMAGE · 습관 3이 값을 하는 곳

# 2클래스인데 sigmoid가 한 번도 안 나와요

```
image_dataset_from_directory(..., label_mode='categorical')
Dense(NUM_CLASSES, activation='softmax')
model.compile(loss='categorical_crossentropy')
```

클래스가 2개든 7개든 **고칠 코드가 없습니다.** 기본값 `'int'` · `'binary'`를 쓰면 클래스 수가 바뀔 때마다 출력층과 손실 함수를 손대야 합니다.

IMAGE · 제일 많이 당하는 실수

# 검증 점수는 멀쩡한데 제출 점수만 랜덤이에요

```
val_ds = image_dataset_from_directory(..., shuffle=True)
preds  = model.predict(val_ds)
filenames = val_ds.file_paths        # ← preds 와 순서가 다르다!
```

```
# 해결 – 순서가 보장되는 예측 전용 데이터셋
val_pred_ds = (tf.data.Dataset.from_tensor_slices(val_ds.file_paths)
               .map(_load_image).batch(BATCH_SIZE))
```

`shuffle=True`는 반복할 때마다 순서가 바뀌는데 `file_paths`는 정렬 순서 그대로입니다. **파일명과 예측이 어긋난 채 제출**되고, 검증 점수는 정상이라 발견이 어렵습니다.

# 시험 당일

180분을 어떻게 나눠 쓸지, 그리고 커널이 하나라서 생기는 일들을 볼게요.

시험 당일 · 순서

# 학습이 긴 문제부터 손대요

| 순서 | 유형 | 배점 | 학습 |
|---|---|---|---|
| **1** | **이미지** (3번 문제) | 35 | 가장 긺 · 먼저 걸어두기 |
| **2** | 텍스트 TF-IDF (2번 문제) | 35 | 몇 초 · 빠르게 확보 |
| **3** | 정형 (1번 문제) | 30 | 1분 미만 |

이유

## 이미지 학습이 가장 길다

텍스트(TF-IDF)·정형은 **거의 즉시** 끝나 먼저 제출물을 확보할 수 있습니다. **이미지 학습을 가장 먼저 걸어 두고**, 그 시간 동안 나머지 둘을 처리하세요. 학습 데이터가 크면 일부만 써서 시간을 줄입니다.

```
# 커널이 하나라는 뜻
이미지 fit() 실행 ──┐  학습 중엔 다른 셀 실행 못 함. 작성은 됨.
학습 종료         ──┘  대기하던 텍스트 · 정형 셀이 이어서 실행
```

시험 당일 · 꼭 기억

# 중간저장 과 GPU 재연결

문항을 풀고 나면

### 중간저장 한 번씩

한 문항 끝낼 때마다 저장하세요. 안 하면 커널이 튕길 때 **코드가 통째로 날아갑니다.**

커널이 재시작되면

### GPU 꼭 다시 연결

세션이 튕기면 GPU 가 떨어져 조용히 CPU 로 붙습니다. **세션 시작 셀부터 다시** 실행해 GPU 를 붙이고 시작하세요.

**이 두 가지를 놓치면 시간과 결과가 통째로 날아갑니다.**

시험 당일 · 변수

# 커널이 하나라<br>전역 변수가 공유돼요

```
# 3번 문제 (이미지) - 먼저 풀었다
X_train = ...

# 1번 문제 (정형) - 같은 커널
X_train = ...   # 이미지의 X_train 이 사라졌다
```

이미지 문제로 돌아가 셀 하나만 다시 돌리면 **에러 없이 틀린 결과**가 나옵니다.

```
p1_X_train = ...   # 1번 Tabular
p2_X_train = ...   # 2번 Text
p3_model   = ...   # 3번 Image
```

문제별 접두사

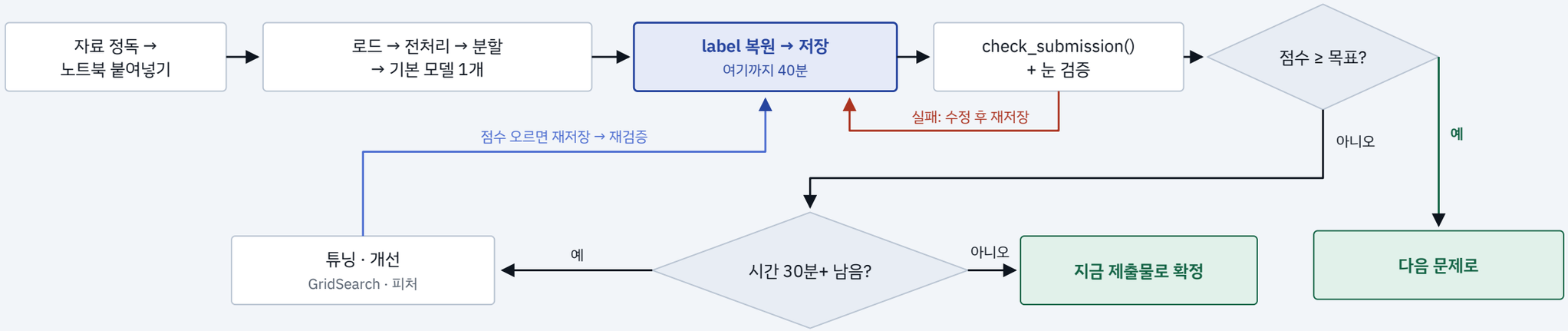
### p1_ · p2_ · p3_ 로 분리

시험 순서가 **1 Tabular · 2 Text · 3 Image**로 고정이니 접두사를 미리 정해 두면 서로 안 덮어씁니다.

제출 파일명의 문제 번호( `_1` · `_2` · `_3` )를 놓치면 **다른 문제 제출물을 덮어써요.**

시험 당일 · 흐름도

# 저장부터 하고 개선은 반복해요




완벽해질 때까지 붙잡는 흐름이 아닙니다 — **일단 저장**하고, 점수와 남은 시간이 허락할 때만 루프를 돕니다. 두 루프 모두 **저장으로 되돌아오는** 것에 주목하세요.

시험 당일 · 지문 읽는 법

# 지문은 4단계예요. ③④ 어기면 0점

① 데이터

### train · test

**test 엔 정답 없음** — 예측해 제출하는 대상. 내부 검증 X_val 과 다름.

② 맞출 것

### 타겟 · 지표

분류/회귀, 요구 지표를 튜닝 scoring 에.

③ 결과물

### 컬럼명 · 값

지문 표기 그대로. 값은 label(0/1 아니라 **Y/N**).

④ 파일명

### 번호_문제.확장자

모델 `_1.pkl` / 결과 `_1.csv`.

```
# ③ 결과물: 모델은 숫자로 예측 → 지문의 컬럼명 · label 로 바꿔 test 원본에 붙인다
pred = p1_target_encoder.inverse_transform(pred)      # [0,1] → ['N','Y']
p1_submission = p1_df_test.copy()                     # 결과 CSV = test 원본 + 결과 컬럼
p1_submission[PRED_COL] = pred
p1_submission.to_csv(SUBMIT_CSV, index=False)
```

제출 전 반드시

### 결과 컬럼명 · 결과 데이터 · 역라벨링 확인

① 결과 **컬럼명**이 지문과 같은지 ② 결과 **데이터(값)**가 맞는지 — 라벨 인코딩했다면 **역라벨링 (inverse_transform)** 돼 원래 값으로 나가는지 눈으로 확인.

③ 컬럼명·값, ④ 파일명은 모델 성능과 무관한 **0점 지대**입니다. 지문 표기를 그대로 옮겨 적어요.

정 리

## 끝까지 도는 코드를 먼저 만들고, 남는 시간에 점수를 올려요.

가장 많이 잃는 곳 1

### test로 fit

검증 점수는 올라가서 발견이 어렵습니다.

2

### 완벽주의

한 문제 0.95보다 두 문제 0.85가 유리합니다.

3

### label 미복원

컬럼명과 값이 요구된 label과 다르면 **0점.** 행 수 · `index=False` · 파일명도.

시험장에서 처음 보는 걸 시도하진 마세요. 노트북을 손에 익혀 두는 것이 알고리즘을 하나 더 아는 것보다 확실합니다. 샘플 문항 링크는 2쪽 '참고자료'에 있습니다.

# Q&A

궁금한 점이나 막혔던 부분, 편하게 물어보세요.

# 고생하셨습니다

노트북 네 벌 손에 익히고 **Colab T4**에서 미리 돌려보면, 시험장에서도 그대로예요. 여러분의 합격을 응원합니다.

`실습 = Colab T4  제출 = 번호_문제.csv + 모델  값은 label 복원`